

F4 计算机系统的状态机模型

在进一步学习如何设计处理器之前, 我们先来了解处理器工作的基本原理. 不过我们暂时不会从数字电路的视角来介绍处理器, 而是通过一个不那么严谨的数学模型来阐释一些和处理器相关的重要概念. 因此, 你可以先暂时忘记上一节学习的数字电路知识了. 处理器和数字电路确实有密不可分的联系, 我们将再下一节介绍如何通过数字电路来实现处理器. 但目前, 你只需要准备纸和笔就足够了.

处理器的组成和工作原理

计算机是一个机械地进行数据处理的装置, 这里的"机械"是指其原理可以很直白地理解, 并无神秘之处. 中央处理器(CPU)作为计算机的大脑, 其实也是这样的装置.

指令及其编码

你应该都接触过电子计算器, 现代计算器至少可以进行加, 减, 乘, 除的四则运算. 处理器的功能远比电子计算器要强大, 作为处理数据的装置, 其处理方式肯定不只一种. 另外, 既然处理器是一个机械地进行数据处理的装置, 它要做何种处理, 就应该是受到某种方式的控制, 而不存在可以自己思考的智能. 这种控制的处理器的媒介, 就是指令. 例如, 我们可以通过加法指令来控制处理器对两个数据相加, 这和我们在电子计算器中输入 `1+2=` 来控制它是非常类似的.

要"用加法指令来控制处理器对两个数据进行相加", 说明一条指令需要给出两方面的信息: 一方面, 计算机需要处理的数据有很多, 因此指令中需要指定需要处理哪些数据, 这称为指令的"操作数"(operand)字段; 另一方面, 计算机处理数据的方式也有很多种, 因此指令中也需要指定用何种方式处理数据, 这称为指令的"操作码"(opcode)字段.

1	+-----+-----+-----+		
2	opcode	operand	
3	+-----+-----+-----+		

💡 生活中的"处理器": 厨房

事实上, 处理器中的很多概念, 都可以在生活中找到相似的例子. 例如, 厨房是一个加工食材的场所, 厨师会根据菜谱对食材进行加工处理, 最终做出一道道菜.

处理器的工作方式和厨房中发生的事件非常类似. 例如, 指令就像是菜单, 厨师按照菜单做菜, 也可以理解成菜单"控制"着厨师做菜. 而菜单中, 也有类似"操作数"和"操作码"的概念. 例如, "番茄炒鸡蛋"中的"番茄"和"鸡蛋"就类似"操作数", 说明这道菜是要对"番茄"和"鸡蛋"这两种食材进行加工处

理; 而"炒"就类似"操作码", 指示了需要对食材进行哪种烹饪操作. 类似回锅肉, 清蒸鲈鱼这些菜单上的菜, 都可以从指令的角度理解它们.

注意到有些数据处理的中间结果应该保存起来供后续使用, 因此处理器中应该有一些部件能够保存处理的数据, 这就是寄存器. 为了方便存放数据, 通常来说这样的寄存器不只一个, 而是由多个寄存器组成一个"寄存器组"(Register Set, 有的教科书也称其为"寄存器堆", 英文为Register File). 由于这样的寄存器组用于处理一般数据, 因此也称其为通用寄存器(General Purpose Register, GPR). 相对地, 有一些寄存器的功能并非用于处理一般处理, 它们不属于GPR.

例如, 要计算 $1+2+\dots+10$, 但一条加法指令只能计算两个数据相加的结果, 因此需要将 $1+2$ 的结果保存在GPR中的某个寄存器 r 中, 然后计算 $r+3$ 等等. 这样, 指令就需要在操作数字段中指定从哪个GPR中读出数据, 以及将计算结果存入哪个GPR中. 至于操作码, 处理器只需要约定好每种操作码代表何种操作即可.

💡 生活中的"处理器": 厨房(2)

"回锅肉"这道菜需要先煮后炒, 因此需要先将煮好的肉临时盛到碗里, 然后再将煮好的肉从碗里倒回锅里翻炒. 处理器中的寄存器就好比是厨房中用来临时存放食材的碗.

处理器作为一个数字电路, 所有信息都可以用 0 和 1 来表示, 指令也不例外. 例如, 某简单处理器有4个GPR, 支持3种指令, 其中2种如下:

1	7	6	5	4	3	2	1	0	
2	+-----+-----+-----+-----+								
3		00		rd		rs1		rs2	R[rd]=R[rs1]+R[rs2] add指令，寄存器相加
4	+-----+-----+-----+-----+								
5		10		rd		imm			R[rd]=imm li指令，装入立即数，高位补0
6	+-----+-----+-----+-----+								

由于指令只有3种, 因此最少可以用2位操作码就能区分所有指令. 在上述例子中, 操作码是 00 , 表示 `add` 指令; 操作码是 10 , 表示 `li` 指令. 为了方便叙述, 指令通常有相应的名称, 如 `add` 指令和 `li` 指令.

关于操作数, 因为GPR只有4个, 因此可以用2位来指定一个GPR的地址(00 , 01 , 10 , 11). 操作数据的来源称为"源寄存器", 上述的 `add` 指令需要从两个源寄存器中获取加数, 两个源寄存器的地址分别记为 $rs1$ 和 $rs2$, 分别用 $R[rs1]$ 和 $R[rs2]$ 来表示GPR中存放的内容; 操作需要写入的目的称为"目的寄存器", 一般记为 rd . 其中, 第2条 `li` 指令的操作数稍有不同, 其源操作数不再是GPR, 而是直接将指令中的 imm 字段解析成一个二进制数来使用, 这种操作数称为"立即数".

综上, 上述指令的长度都是8位. 我们可以根据上述的指令编码规则来理解一条指令的含义, 一些具体的指令例子如下:

1	00100001	add指令, 将R[0]和R[1]相加, 结果写入R[2]
2	10110000	li指令, 将立即数0000写入R[3]
3	10000101	li指令, 将立即数0101写入R[0]

存储程序

事实上, 在我们认识中几乎无所不能的计算机, 就是靠不断执行一条条指令来完成各种复杂任务的. 你在计算机上运行的各种软件程序, 最终也是在执行一条条的指令. 因此, 程序的本质就是一段指令序列. 说起来你可能不信, 毕竟上面这些指令的行为非常简单, 和电子计算器没什么区别, 那计算机凭什么能比电子计算器强大那么多呢?

答案在于计算机的工作机制: 它可以让程序来自动控制计算机的执行. 具体地, 我们可以先把一段指令序列放在存储器(如现代计算机的内存)中, 让计算机从内存中取出指令来执行; 重要的是, 当计算机执行完一条指令之后, 就继续执行下一条指令. 为了能让计算机知道下一条指令在哪里, 还需要有一个用于指示当前执行到哪条指令的部件, 这个部件称为“程序计数器”(Program Counter, PC).

从此以后, 计算机只需要做一件事情:

1	重复以下步骤:
2	从PC指示的存储器位置取出指令
3	执行指令
4	更新PC

这样, 我们只要将一段指令序列放置在存储器中, 然后让PC指向第一条指令, 计算机就会自动执行这一段指令序列, 永不停止. 这就是“存储程序”(Stored-Program)的基本思想. 如今的各种主流计算机, 本质上都是“存储程序”计算机.

还有更巧妙的设计! 既然PC存储了当前执行指令的位置, 那PC也应该是一个寄存器, 这样的话, 我们还可以设计相应的指令来修改它, 从而增加程序执行过程的灵活性. 例如, 我们可以在上文提到的2条指令的基础上, 再添加第3条指令:

1	7 6 5	2 1 0
2	+---+---+---+---+	
3	11 addr rs2	if (R[0]!=R[rs2]) PC=addr bner0指令, 若不等于R[0]则跳转
4	+---+---+---+---+	

这条 `bner0` 指令十分特殊, 它是 `Branch if Not Equal r0` 的缩写, 如果执行这条指令的时候 `R[rs2]` 与 `R[0]` 不相等, 则将PC寄存器更新为 `addr`, 即让PC指向 `addr` 处的指令. 不过, PC寄存器并不是用于处理一般数据, 因此它不属于GPR.

有了上面这些指令, 我们就可以让计算机做一些不简单的事情了.

一个数列求和的例子

让我们用指令来计算 $1+2+\dots+10$ 这一数列的和. 为了方便理解, 我们先不采用 0 和 1 来表示指令. 我们用 `r0`, `r1`, `r2` 和 `r3` 分别指代4个GPR, 并且用逗号来分隔指令的操作数. 假设以下指令序列存放在存储器中, 用于计算上述数列之和, 其中 : 前的数字表示PC, # 及其后的文字表示注释:

```
1    0: li r0, 10    # 这里是十进制的10
2    1: li r1, 0
3    2: li r2, 0
4    3: li r3, 1
5    4: add r1, r1, r3
6    5: add r2, r2, r1
7    6: bner0 r1, 4
8    7: bner0 r3, 7
```

乍眼一看, 你可能很难理解上面的指令序列如何实现数列求和. 让我们来把自己当作处理器, 通过执行一条条指令来理解这个过程. 为了理解指令执行的过程, 我们还需要记录寄存器的变化. 我们用 (PC, r0, r1, r2, r3) 的格式来记录寄存器的值, 这一格式也反映了处理器所处的状态. 例如 (7, 2, 3, 1, 8) 表示接下来将要执行编号为 7 的指令, 当前4个GPR的值分别为 2, 3, 1, 8. 我们约定在开始的时刻, 处理器的状态是 (0, 0, 0, 0, 0). 以下是处理器执行前若干条指令的过程:

```
1    PC r0 r1 r2 r3
2    (0, 0, 0, 0, 0) # 初始状态
3    (1, 10, 0, 0, 0) # 执行PC为0的指令后, r0更新为10, PC更新为下一条指令的位置
4    (2, 10, 0, 0, 0) # 执行PC为1的指令后, r1更新为0, PC更新为下一条指令的位置
5    (3, 10, 0, 0, 0) # 执行PC为2的指令后, r2更新为0, PC更新为下一条指令的位置
6    (4, 10, 0, 0, 1) # 执行PC为3的指令后, r3更新为1, PC更新为下一条指令的位置
7    (5, 10, 1, 0, 1) # 执行PC为4的指令后, r1更新为r1+r3, PC更新为下一条指令的位置
8    (6, 10, 1, 1, 1) # 执行PC为5的指令后, r2更新为r2+r1, PC更新为下一条指令的位置
9    (4, 10, 1, 1, 1) # 执行PC为6的指令后, 因r1不等于r0, 故PC更新为4
10   (5, 10, 2, 1, 1) # 执行PC为4的指令后, r1更新为r1+r3, PC更新为下一条指令的位置
11   .....
```

继续执行上述指令

尝试继续执行指令, 记录寄存器的状态变化过程. 你发现执行到最后时, 处理器处于什么样的状态? 上述数列的求和结果在哪个寄存器中?

可以看到, 处理器的工作过程就是按照指令的含义机械地更新寄存器的状态. 到此为止, 你已经理解处理器工作的基本原理了.

事实上, 计算机的优势在于其极快的运算速度: 现代处理器的主频已经到GHz量级了, 以2GHz为例, 这意味着处理器在1秒内会经过2000000000个时钟周期. 而用上述指令序列计算 `1+2+...+10` 只需要不到40条指令, 如果按每周期执行1条指令来计算, 现代处理器只需要花费0.00000002s, 即20ns, 即可完成计算. 如果我们人工按电子计算器, 1秒按10个键已经算是手速非常快了, 但输入 `1+2+...+10=` 需要按22次按键, 算下来也需要2s. 如果要计算 `1+2+...+10000`, 我们只需要将上述PC为0的指令改为 `li r0, 10000` 即可 (不过需要更长的指令来表示 `10000` 这个立即数), 按同样的方式计算, 需要执行约30000条指令, 也只需要花费0.000015s, 即15us. 如果人工按电子计算器, 则需要按48894次按键, 算下来需要4889s! 即使我们用等差数列的求和公式心算 ($(10000 + 1) * 10000 / 2$), 也需要若干秒才能算出正确答案, 和强大的计算机相比可谓是相形见绌.

重新审视编程

你之前可能接触过编程语言, 但应该很少思考编程的本质. 所谓编程, 实际上是利用给定功能的组合来完成复杂任务的过程. 大家可能接触过C++, Python, Scratch等编程语言, 这些都属于高级编程语言. 而上述通过指令的组合来实现数列求和的例子, 其实也是编程!

上述的 `add` 和 `li` 等指令, 在计算机领域里面属于汇编语言的范畴. 汇编语言是指令的符号化表示. 与汇编语言相比, 还有更底层的机器语言, 它就是指令的二进制表示, 可以被通过数字电路实现的处理器直接执行. 例如, 上述数列求和程序的机器语言表示如下:

1	10001010	# 0: li r0, 10
2	10010000	# 1: li r1, 0
3	10100000	# 2: li r2, 0
4	10110001	# 3: li r3, 1
5	00010111	# 4: add r1, r1, r3
6	00101001	# 5: add r2, r2, r1
7	11010001	# 6: bner0 r1, 4
8	11011111	# 7: bner0 r3, 7

事实上, 上述机器语言表示就是根据前文约定的指令编码将汇编语言翻译成 `0` 和 `1` 的序列. 只要了解指令的编码规则, 汇编语言和机器语言可以互相转换. 不过如果只看机器语言, 程序员是很难理解的; 相比之下, 汇编语言能更直观地表示指令的操作码和操作数, 可读性比机器语言更好.

计算10以内的奇数之和

尝试用上述指令编写一个程序, 求出10以内的奇数之和, 即计算 `1+3+5+7+9`. 编写后, 尝试列出处理器状态的变化过程, 以此来检查你编写的程序是否正确.

指令集架构的状态机模型

上文介绍的GPR, PC, 存储器, 指令及其执行过程, 在计算机领域中属于指令集架构(Instruction Set Architecture, 缩写为ISA, 也简称指令集)的范畴. 你可能听说过x86, ARM, RISC-V等概念, 它们其实都是ISA. 上文提到的那个只有3条指令的例子, 其实也是一种ISA, 为了方便叙述, 我们将其称为sISA(simple ISA).

ISA的本质是一系列规范, 这些规范通常记录在相应的手册中, 它们定义了一台模型机的功能和行为. 所谓的模型机就是一台只存在于思维中的机器, 我们只讨论其具备的功能和行为, 而不讨论其具体实现. 因此, 围绕模型机开展学习可以将原理和实现细节分离, 有助于大家理解处理器的本质. 等到我们把这台思维中的模型机通过数字电路实现出来, 我们就得到了一台真正的计算机.

在进一步解释ISA中具体包含什么之前, 我们先来介绍一个对我们理解计算机系统非常重要的数学模型: 状态机.

状态机的定义包含如下部分:

- 状态集合 $S = \{S_1, S_2, \dots\}$
- 激励事件集合 E
- 状态转移规则 $next : S \times E \rightarrow S$
 - 描述每个状态在不同激励事件下的次态(next state), 即二元函数 $next(S, E)$ 给出了在状态 S 下接收到激励事件 E 后的次态
- 初始状态 $S_0 \in S$

虽然上述定义在数学意义上并非100%严谨, 但这对我们理解计算机系统的行为来说已经足够了.

现在我们来从状态机的视角去理解ISA:

- 状态集合. 回忆时序逻辑电路的相关内容, 状态是那些可以稳定存储信息的元素. 对这个含义进行引申, 在ISA中, 状态应该包含PC, GPR和内存. 也即, ISA中的一个状态是一组具体的PC, GPR和内存, 而全体状态的集合 S 则是PC, GPR和内存所有取值的组合, 即 $S = \{(PC, R, M)\}$.
- 激励事件集合. 在ISA中, 执行指令会改变状态, 因此执行指令就是这个状态机的激励事件.
- 状态转移规则. 按照定义, 在ISA中, 状态转移规则用于描述“在某个状态下执行某指令后的次态”, 也即指令的语义, 它约定了执行某指令后, 状态应该发生怎么样的变化, 从而从一个状态转移到另一个.
- 初始状态. 在未执行任何指令之前的状态.

以上述的sISA为例, 其状态机模型如下:

- 状态集合 $S = \{(PC, R, M)\} = \{(PC, r0, r1, r2, r3, M)\}$. 例如, 在数列求和例子中, 某个状态可表示为

$$S_k = (5, 10, 1, 0, 1, [10001010, 10010000, 10100000, 10110001, 00010111, 00101001, 11010001, 11011111])$$

其中 $[]$ 表示内存中存储的信息, 这里就是上文指令序列的编码. 不过由于在sISA中, 指令并不会修改内存中存储的信息, 为了简化表示, 我们可以省略状态中对内存的描述: $S_k = (5, 10, 1, 0, 1)$.

- sISA作为一种具体的ISA, 激励事件同样是执行指令.
- 在sISA中, 状态转移规则就是以下3条指令的语义:

1	7 6 5 4 3 2 1 0	
2	+-----+	
3	00 rd rs1 rs2 R[rd]=R[rs1]+R[rs2]	add指令, 寄存器相加
4	+-----+	
5	10 rd imm R[rd]=imm	li指令, 装入立即数, 高位补0
6	+-----+	
7	11 addr rs2 if (R[0]!=R[rs2]) PC=addr bner0指令, 若不等于R[0]则跳转	
8	+-----+	

例如, $S_{k+1} = next(S_k, 00101001)$ 表示, 在状态为 $S_k = (5, 10, 1, 0, 1)$ 时执行指令 `00101001` 后的次态为 S_{k+1} . 根据上文, 该指令的行为是 `R[2] = R[2] + R[1]`, 因此 $S_{k+1} = (6, 10, 1, 1, 1)$. 这其实是上述数列求和例子执行过程中的其中一步.

- 初始状态 $S_0 = (0, 0, 0, 0, 0)$.

人生如戏

我们在上文中已经用自然语言和数列求和的例子解释了计算机的工作原理. 我们之所以还要给大家介绍充满数学符号的状态机模型, 是为了帮助大家揭开计算机的神秘面纱: 计算机自从诞生以来, 其工作过程就是一个数学游戏, 状态机模型只不过是这个数学游戏的规则.

事实上, 无论是上面的指令集, 还是后面我们要介绍的程序和处理器, 这些计算机的核心概念在本质上都是一致的: 它们都是状态机这个数学游戏. 因此, 如果你已经掌握了上面的游戏规则, 你就已经做好了理解计算机的准备.

而对于x86, ARM, RISC-V这些商业级别的真实ISA, 它们的GPR数量更多, 指令数量更多, 指令行为也更复杂. 但其本质原理和上述只有3条指令的sISA并没有本质区别, 因此, 如果你将来需要理解它们, 你只需要阅读相应的手册, 了解更多指令的细节即可.

ISA的更多内容

除了指令的语义之外, ISA还包括输入输出, 系统状态, 中断异常, 虚存管理, 内存模型. 随着学习进度的深入, 我们会依次向你介绍这些概念, 你也有机会将这些概念实现到自己设计的处理器中, 让处理

C程序入门

上文介绍了汇编语言, 虽然采用汇编语言也可以编程, 但如果要开发更大的程序, 汇编语言就不方便了. 这是因为用汇编语言编程需要关注数据如何在GPR之间流动, 而且一条指令能进行的操作普遍很少, 即使是一段简单的程序逻辑, 可能都需要十几条指令才能表达. 事实上, 现代的程序开发通常采用高级编程语言.

高于编程语言有很多, 但我们选择C语言, 因为它在高级编程语言中属于比较简单的一类, 我们可以很容易将C语言与计算机系统的工作过程建立关联, 从而帮助我们理解计算机的本质.

一个简单的C语言示例

让我们先来看一个简单的代码示例:

```
1  #include <stdio.h>
2
3  /* 1 */ int main() {
4  /* 2 */   int x = 1;
5  /* 3 */   int y = 2;
6  /* 4 */   int z = x + y;
7  /* 5 */   printf("z = %d\n", z);
8  /* 6 */   return 0;
9  /* 7 */ }
```

这个示例将计算 `1+2`, 并输出计算结果. 我们对代码的细节含义进行一些说明:

- 在 `/*` 和 `*/` 之间的内容是C语言的注释(comment), 注释内容不影响程序的逻辑, 因此注释主要是给程序员阅读的. 例如, 上述代码和以下代码的逻辑是等价的:

```
1  #include <stdio.h>
2
3  /* This is
4     an example. */
5  int main() {
6     int x = 1;
7     int y = 2;
8     int z = x + y;
9     printf("z = %d\n", z);
10
11 }
```



```
return 0;
}
```

- `int` 表示整数类型, 它是 `integer` 的缩写.
- `int main() { ... }` 定义了一个名称为 `main` 的函数. 函数是C语言的基本模块, 执行C程序的过程, 就是执行C程序中函数代码的过程. 特别地, `main` 函数是一个特殊的函数, 它是C程序的入口, 也即, C程序将从 `main` 函数开始执行.
- 大括号中的内容 `{ ... }` 是函数体, 由语句组成, 每条语句用于指示程序执行一个操作. 大括号中的语句默认按顺序执行.
- `int x = 1;` 定义了一个变量, 变量的名称为 `x`, 并将其初值赋为 `1`. 变量是程序中存储信息的对象, 类似上文提到的GPR. 在C语言中, 一条语句以分号 `;` 结束. `int y = 2;` 同理.
- `int z = x + y;` 定义了一个变量 `z`, 并将其初值赋为 `x + y`.
- `printf("z = %d\n", z);` 是一个函数调用语句, 它调用了函数 `printf`. 程序中的函数调用和数学上的求函数值有一些相似之处, 例如在数学领域中, 已知 $f(x) = x + 1$, 求 $f(3)$ 的过程就是将 `3` 代入 f 的定义中进行计算的过程; `printf("z = %d\n", z);` 就是将两个参数 `"z = %d\n"` 和 `z` 代入 `printf` 的定义中并执行. 在C程序中, `printf` 是一个特殊的函数, 它用于向终端输出信息, 第一个参数中的 `%d` 表示将第二个参数 `z` 按十进制格式输出. 因此, 上述函数调用会将变量 `z` 的当前值按十进制格式输出到终端.
- `return 0;` 是函数返回语句, 表示该函数执行结束, 并将 `0` 作为计算结果返回给调用该函数的语句.

在这个例子中, 我们已经介绍了C语言中一些关键的组成要素:

1. 变量, 它们是C程序处理的对象
2. 语句, 它们是C程序处理的操作流程
3. 输入输出函数, 它们是C程序中让变量与外界交互的桥梁

在线运行C程序

你可以在 [Compiler Explorer](#) 这个在线网站中运行上述C程序. 具体地, 在代码编辑器中输入上述C代码后, 点击编辑器上方工具栏的 `Add new...`, 选择 `Execution Only`, 即可弹出执行结果的终端窗口. 如果你的操作正确, 你将看到终端窗口输出 `z = 3`.

你可以尝试按照你的理解修改C代码, 并观察程序的输出.

Hint: 你可以收藏这个网站, 在E阶段的前期, 你将会使用这个网站作为开发环境.

需要注意的是, 上述说明并未详尽所有相关的C语言细节, 部分细节目前无法解释清楚, 我们先省略它们. 不过, 以上说明已经足够帮助你理解讲义的后续内容了.

C程序的状态机模型

也许你已经隐约感觉到, C语言中的变量和语句的概念, 和上文提到ISA的GPR和指令的概念, 是有那么一点相似之处. 事实上, 从计算机系统的视角来看, 它们之间还真的具有十分密切的关系. 和ISA类似, 我们也可以从状态机的视角来理解C程序:

- 状态集合. 在C程序中, 最直接的状态就是变量, 因为它们是存储信息的对象. 除此之外, 我们在上文还提到: 大括号中的语句默认按顺序执行 . 很自然地, 在C程序的执行过程中, 还需要有一个用于指示当前执行到哪条语句的隐含变量, 这个隐含变量就是C语言的“程序计数器”(PC), 它也应该属于状态的一部分. 因此, 在C程序中, 一个状态是一组具体的变量和PC, 而全体状态的集合 S 则是变量和PC所有取值的组合, 即 $S = \{(PC, V)\}$, 其中 V 表示程序中所有变量的取值.
- 激励事件集合. 在C程序中, 执行语句会改变状态, 因此执行语句就是这个状态机的激励事件.
- 状态转移规则. 按照C程序的行为, 状态转移规则用于描述“在某个状态下执行某语句后的次态”, 也即语句的语义, 它约定了执行某语句后, 状态应该发生怎么样的变化, 从而从一个状态转移到另一个.
- 初始状态. 在未执行任何语句之前的状态, 即 $S_0 = (main函数第一句条语句的位置, V_0)$.

给出上述状态机模型之后, 我们就可以从状态机的视角来理解C程序的执行过程了. 考虑上述示例代码, 我们让PC表示程序执行到哪一行, 行号已经通过注释的方式给出. 我们用 (PC, x, y, z) 的格式来记录上述程序的状态. 根据C语言的约定, 初始状态为 $(2, ?, ?, ?)$, 其中PC为 2 表示从第2行开始执行, 即 `main` 函数的第一条语句, `?` 表示变量还未定义或未赋初值. 以下是上述C程序的执行过程:

1	PC	x	y	z	
2	(2, ?, ?, ?)				# 初始状态
3	(3, 1, ?, ?)				# 执行PC为2的语句后, x更新为1, PC更新为下一条语句的位置
4	(4, 1, 2, ?)				# 执行PC为3的语句后, y更新为2, PC更新为下一条语句的位置
5	(5, 1, 2, 3)				# 执行PC为4的语句后, z更新为x + y, PC更新为下一条语句的位置
6	(6, 1, 2, 3)				# 执行PC为5的语句后, 终端输出"z = 3", PC更新为下一条语句的位置
7	(END, 1, 2, 3)				# 执行PC为6的语句后, 从main函数返回, 程序执行结束

用C语言实现数列求和

我们还可以用C语言来计算 `1+2+...+10`, 考虑以下C程序:

```
1  #include <stdio.h>
2
3  /* 1 */ int main() {
4  /* 2 */   int sum = 0;
5  /* 3 */   int i = 1;
6  /* 4 */   do {
7  /* 5 */       sum = sum + i;
8  /* 6 */       i = i + 1;
9  /* 7 */   } while (i <= 10);
10 /* 8 */   printf("sum = %d\n", sum);
11
```

```
12      /* 9 */   return 0;
      /* 10*/ }
```

上述代码中包含一个 `do-while` 循环, 其行为是, 先执行循环体(在大括号中)中的语句, 然后判断循环条件(在小括号中)是否成立, 若成立, 则重复执行循环体; 否则将结束循环, 继续执行后续语句。

和上文的汇编语言相比, 我们至少看到了用C语言开发程序的两点优势:

1. 变量的命名可以更直观地反映出其用途, 但汇编语言中, GPR的用途只能根据上下文推断
2. 循环的表达更清晰, 可以直接区别循环条件和循环体, 但在汇编语言中, 循环条件和循环体都是指令, 需要根据上下文推断

根据上述程序的行为, 前若干条语句的执行过程如下:

1	PC	sum	i	
2	(2, ?, ?)			# 初始状态
3	(3, 0, ?)			# 执行PC为2的语句后, sum更新为0, PC更新为下一条语句的位置
4	(5, 0, 1)			# 执行PC为3的语句后, i更新为1, PC更新为下一条语句的位置(第4行无有效操作, 跳过)
6	(6, 1, 1)			# 执行PC为5的语句后, sum更新为sum + i, PC更新为下一条语句的位置
7	(7, 1, 2)			# 执行PC为6的语句后, i更新为i + 1, PC更新为下一条语句的位置
8	(5, 1, 2)			# 执行PC为7的语句后, 由于循环条件i <= 10成立, 因此重新进入循环体
				

继续执行上述程序

尝试继续执行上述代码, 记录状态的变化过程. 程序执行结束时, 程序处于什么样的状态?

数字电路的状态机模型

我们在上文提到"把这台思维中的模型机通过数字电路实现出来", 这意味着, 数字电路也能和ISA建立某种关联, 那我们是否也能从状态机的视角去理解数字电路的行为呢?

答案是肯定的! 根据上一节数字电路的知识, 我们可以很容易地归纳出数字电路的状态机模型:

- 状态集合. 在数字电路中, 有时序逻辑电路才能存储信息, 因此一个状态是时序逻辑元件所存储的具体信息, 而全体状态的集合 S 则是时序逻辑元件所能存储信息的所有组合.
- 激励事件集合. 既然时序逻辑元件表征了数字电路的状态, 而时序逻辑元件的内部状态可以通过其输入端改变(例如可以通过输入端将数据写入D触发器), 我们可以将数字电路看成以下模型:

```

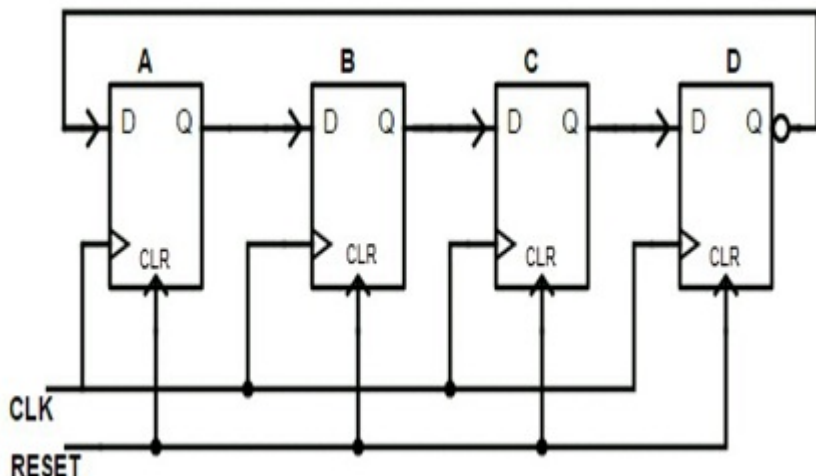
1      +-----+
2  +-->| Sequential Logic |----+
3      | +-----+      |
4      | next state      | current state
5      |                  |
6      | +-----+      |
7  +--| Combinational Logic |<--+
8      +-----+

```

也即, 让时序逻辑元件的状态发生变化的, 其实是组合逻辑电路输出的信号, 因此组合逻辑电路就是这个状态机的激励事件.

- 状态转移规则. 时序逻辑元件的状态具体应如何变化, 是由组合逻辑电路的具体逻辑决定的.
- 初始状态. 即电路在复位时, 时序逻辑元件的状态.

例如, Johnson计数器的电路结构如下:



根据电路结构的逻辑, 我们可以从状态机视角来理解上述Johnson计数器的工作过程:

	A	B	C	D	
1					
2	(0, 0, 0, 0)				# 初始状态
3	(1, 0, 0, 0)				
4	(1, 1, 0, 0)				
5	(1, 1, 1, 0)				
6	(1, 1, 1, 1)				
7	(0, 1, 1, 1)				
8	(0, 0, 1, 1)				
9	(0, 0, 0, 1)				
10	(0, 0, 0, 0)				# 与初始状态一致

在上一小节中, 你已经通过寄存器和加法器搭建出一个简单数列求和电路, 用于计算 $1+2+\dots+10$. 尝试列出电路状态的变化过程.

当然, 这个Johnson计数器的例子与CPU区别很大, 但如果CPU能用数字电路设计出来, 它一定也包含时序逻辑电路和组合逻辑电路, 因而也一定能从状态机视角来理解CPU的工作过程.

在计算机上执行C程序

我们从状态机视角去理解C程序的执行过程, 本质上是在我们的思维中执行这个C程序. 但计算机只能执行指令, 它无法理解C程序的含义, 因此也无法执行类似 `sum = sum + i;` 这样的代码. 那怎么让计算机执行C程序呢? 答案是, 将C程序的代码翻译成行为等价的指令序列, 然后让计算机的电路来执行这个指令序列.

编译 = 将C程序翻译成指令序列

这个过程在计算机系统有一个特殊的名称, 叫"编译"(compile). 从原则上讲, 编译的工作可以人工进行, 但现代程序的规模很大, 人工进行编译是非常繁琐的, 因此通常由一类特殊的程序来开展编译工作, 这类特殊的程序称为"编译器"(compiler).

既然执行C程序和执行指令序列都可以从状态机视角去理解, 我们也可以尝试去了解编译器需要完成哪些工作. 我们先将两个状态机的内容列出来(此处暂不关心初始状态):

	C程序	ISA
状态	$\{PC, V\}$	$\{PC, R, M\}$
激励事件	执行语句	执行指令
状态转移规则	语句的语义	指令的语义

因此, 编译器的工作就是将C程序的状态机翻译成ISA的状态机, 也即, 构造

$$\begin{aligned} compile_S &: \{PC, V\} \rightarrow \{PC, R, M\} \\ compile_E &: \{\text{语句}\} \rightarrow \{\text{指令序列}\} \end{aligned}$$

使得

$$compile_S(next(S_{C,k}, \text{语句})) = next(compile_S(S_{C,k}), compile_E(\text{语句}))$$

从而有

$$\begin{aligned}
& compile_S(S_{C,k+1}) \\
= & compile_S(next(S_{C,k}, \text{语句})) \\
= & next(compile_S(S_{C,k}), compile_E(\text{语句})) \\
= & next(S_{ISA,k}, \text{指令序列}) \\
= & S_{ISA,k+1}
\end{aligned}$$

其中, $S_{C,k}$ 表示C语言状态机的第 k 个状态, $S_{ISA,k}$ 表示ISA状态机的第 k 个状态. 用自然语言来描述, 则是: C 程序执行一条语句后的状态, 与ISA执行编译得到的指令序列后的状态, 语义上是等价的.

具体地, 对比两个状态机后可以得知, 编译器需要完成以下工作:

- 将C程序的状态翻译成ISA的状态, 也即
 - 将C程序的PC对应到ISA的PC
 - 将C程序的变量对应到ISA的GPR或内存
- 将C程序的状态转移规则翻译成ISA的状态转移规则, 也即, 将语句翻译成指令序列

结合数列求和的例子理解编译器的工作

我们已经给出了数列求和的C程序和相应的指令序列, 尝试从状态机的视角理解它们之间的联系.

在Compiler Explorer中理解编译器的工作

具体地, 点击编辑器上方工具栏的 `Add new...`, 选择 `Compiler`, 即可弹出相应汇编代码的窗口. 汇编代码默认采用x86指令集, 你不一定能理解每一条指令的具体含义, 但通过背景颜色的高亮和鼠标移动, 你可以看到C程序片段和汇编代码片段之间的对应关系.

CPU设计 = 根据ISA设计数字电路

同样地, 既然ISA和数字电路都可以从状态机视角去理解, 我们也可以尝试去了解CPU设计应该如何开展. 我们先将两个状态机的内容列出来(此处暂不关心初始状态):

	ISA	数字电路
状态	$\{PC, R, M\}$	时序逻辑电路
激励事件	执行指令	处理组合逻辑
状态转移规则	指令的语义	组合逻辑电路的逻辑

因此, CPU设计的工作就是用数字电路的状态机实现ISA的状态机, 也即, 构造

$$CPUdesign_S : \{PC, R, M\} \rightarrow \{\text{时序逻辑电路}\}$$
$$CPUdesign_E : \{\text{指令}\} \rightarrow \{\text{组合逻辑电路}\}$$

使得

$$CPUdesign_S(next(S_{ISA,k}, \text{指令})) = next(CPUdesign_S(S_{ISA,k}), CPUdesign_E(\text{指令}))$$

从而有

$$\begin{aligned} & CPUdesign_S(S_{ISA,k+1}) \\ = & CPUdesign_S(next(S_{ISA,k}, \text{指令})) \\ = & next(CPUdesign_S(S_{ISA,k}), CPUdesign_E(\text{指令})) \\ = & next(S_{CPU,k}, \text{组合逻辑电路}) \\ = & S_{CPU,k+1} \end{aligned}$$

其中, $S_{CPU,k}$ 表示CPU电路状态机的第 k 个状态. 用自然语言来描述, 则是: ISA执行一条指令后的状态, 与设计出的CPU在组合逻辑电路控制下的次态, 语义上是等价的.

具体地, 对比两个状态机后可以得知, CPU设计需要完成以下工作:

- 用数字电路的状态实现ISA的状态, 也即, 用时序逻辑电路实现PC, GPR和内存
- 用数字电路的状态转移规则实现ISA的状态转移规则, 也即, 用组合逻辑电路实现指令的功能

程序, ISA和CPU之间的联系

到此, 我们可以来简单梳理程序, ISA和CPU之间的联系:

1. 根据ISA手册的功能描述, 画一张CPU的结构图 -> 处理器微结构设计
2. 根据结构图设计具体的电路 -> 逻辑设计
3. 开发程序 -> 软件编程
4. 将程序翻译成ISA手册中描述的指令序列 -> 编译
5. 在CPU上执行程序 = 用程序编译出的指令序列控制CPU电路进行状态转移
 - 此时, 三个状态机产生联系: $S_C \sim S_{ISA} \sim S_{CPU}$

🔊 一句话就能说清楚, 为什么搞那么复杂?

这是因为, 状态机模型是理解复杂系统的一种有效方法, 有些概念和问题从状态机模型思考可以给我们带来新的启发. 随着"一生一芯"的学习的深入, 我们还会使用状态机模型来分析一些关键的问题, 并提出一些有效的解决方法.

不过, 最重要的是给大家传达这样一种观念: **机器永远是对的**. 计算机系统的行为是按照官方手册的描述精确发生的, 系统的每一次状态转移都有手册依据. 换句话说, 如果你不理解计算机系统的行为, 很大概率是因为你不了解相关手册中的某些关键细节.

你在刚学习"一生一芯"的时候可能会感受到未知的事物像潮水一般袭来. 但只有树立正确的观念, 才有助于解决问题. 有的初学者会提出"是不是操作系统有问题", "是不是编译器有bug"等疑问, 这些不专业的想法并不能帮助你真正解决问题, 不能帮助你获得真正的成长.

当然, 这里的"永远"并不是说不能质疑机器. 事实上, 商业产品也会有bug, 例如历史上著名的Intel奔腾的浮点除法bug^[1]. 但专业人士在质疑的时候, 都会拿出证据, 来证明自己的质疑并非无中生有. 而且越是不可能发生的问题, 求证过程越谨慎. 你所使用的计算机系统大概率是具有一定成熟度的, 你恰好遇上bug的概率本身就很小; 如果你拿不出让人信服的证据, 机器出错的概率就更低了.

● 从一到无穷大

你或许觉得上文举的示例过于简单, 我们使用的现代计算机真的是这样工作的吗? 现在我们来揭开现代计算机的神秘面纱.

我们已经知道, 通过使用补码表示, 计算机可以借助同一套加法器硬件来实现算术上的加法和减法, 通过提供一条新的减法指令, 程序就可以进行减法操作了. 通过跳转指令实现的循环来重复执行加法指令, 我们就实现了算术上的乘法功能; 而重复执行减法指令, 则可以实现算术上的除法功能.

上面已经实现了整数的四则运算, 要实现小数的运算也不难. 我们知道在数学中可以通过科学记数法来表示一个小数, 计算机也可以用类似的方式来表示小数, 只不过采用的是二进制的科学记数法. 具体地, 任何一个小数 x 都可以表示成 $1.\alpha \times 2^\beta$, 其中 α 是二进制表示的无符号数, β 是二进制表示的有符号数. 计算机只要将 α 和 β 记录下来, 就相当于通过二进制方式表示了小数 x . 显然, 小数之间的四则运算也可以转换成 α 和 β 相关的四则运算.

实现了小数的四则运算后, 就可以计算数学意义上的基本初等函数了, 包括常函数, 幂函数, 指数函数, 对数函数, 三角函数, 反三角函数. 具体地, 除常函数外, 其他5种基本初等函数都可以通过幂级数展开进行计算, 而幂级数可以通过四则运算和循环进行计算.

实现了基本初等函数的计算后, 就可以通过基本初等函数之间的有限次有理运算和复合操作, 来计算所有初等函数了. 对于复合操作, 我们可以通过程序中的函数调用功能来实现.

我们还能继续对函数进行求导和积分. 根据导数的定义, 我们只需要代入一个很小的 Δx , 就能近似计算出函数在某一点的导数. 而根据定积分定义, 我们可以将被积区间分成足够多份, 然后用被积函数在区间内的某点取值近似代替区间中的取值, 然后通过求和计算出函数在被积区间上的黎曼和. 当然, 这些都是简单直观的计算方法, 为了在计算机中计算出更精确的导数和积分, 科学家还提出了一系列的数值分析算法.

上述计算都是在实数域上进行, 要将计算扩展到复数域其实也不难. 具体地, 我们只需要对实数分量和虚数分量分别进行计算, 另外再应用相关的规则处理虚数单位 i 即可. 尽管复数域的有些操作和实数域有所不同, 但只要能用数学语言描述出计算方式, 都有办法将其归约到上文的操作中.

支持复数域的求导和积分后,我们就可以在程序中实现各种物理引擎,对现实世界中的各种物理规律进行模拟.各种力学,电磁学,光学,热学的计算公式,在程序中都是通过一条条简单的指令来完成计算,从而让用户在计算机世界中感受到和物理世界一致的体验.我们在计算机中感受到的一切,都是通过计算实现的!

当然,为了让计算过程更高效,现代ISA还提供了更多的指令,包括乘除指令,逻辑运算指令(计算与,或,非等),浮点指令,原子指令等.例如,相比于通过循环实现乘法,计算机提供了乘法指令和乘法器电路,可以让程序通过乘法指令高效地计算乘法.一些ISA甚至还提供直接计算初等函数的指令,如开方指令,三角函数指令等.不难想象,要在电路层次提供计算这些初等函数的功能,是需要电子工程师仔细考量的.

除了提供更丰富的指令,计算机架构师还致力于设计出能更高效执行指令的计算机.例如,通过流水线技术可以提升计算机执行指令的吞吐,通过超标量技术可以让计算机在一个周期内执行多条指令,通过缓存技术则可以提升计算机访问内存的效率,通过GPU等处理器加速特定任务的执行效率... 这些技术最终都会让计算机用户获得越来越快的性能体验.

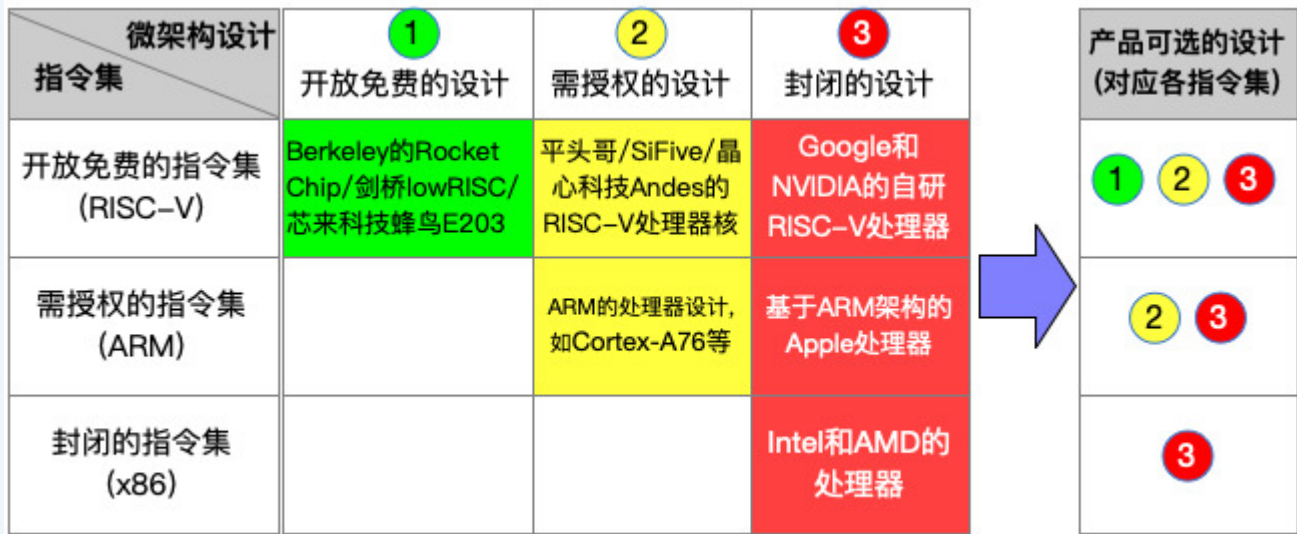
🗨 指令集和处理器是不同层次的概念

随着RISC-V的流行,越来越多人了解到处理器芯片的相关概念.不过由于"指令集"是一个相对抽象的概念,不少非专业人士并不能完全理解,从而得出了不符合实际情况的结论.

事实上,指令集和处理器是不同层次的概念,前者是规范,后者是具体实现.两者各自有3种知识产权模式:

1. 开放免费(Open & Free), 不花钱就能用
2. 可授权(Licensable), 花钱就能用
3. 封闭(Closed), 花钱也不能用

两个层次各自的3种知识产权模式,可以组合出一个3x3的矩阵.下图出自《关于RISC-V和开源处理器的一些解读》[🔗](#).



* 此图源于David Pattern教授在 2019年ISCA ACM SIGARCH Visioning Workshop上的报告《A Golden Age for Computer Architecture》

从图中可以得出两个结论:

1. 封闭指令集不可能做出开源处理器, 因此矩阵的左下三角都是空白
2. 开放指令集并不意味着做出的处理器必须开源, 因此矩阵的右上三角并非空白

第2个结论足够说明, 指令集和处理器是两个不同层次的概念. 对非专业人士来说, 它们之所以不易区分, 一个原因是过去只有可授权和封闭这两种模式, 非专业人士大多只看到矩阵的对角线, 从而将指令集和处理器划上等号.

事实上, 无论是1941年平均每秒执行 3×10^{-1} 次乘法操作的Z3计算机, 还是2024年11月在Top500排名第一, 峰值计算能力达到每秒 2.746×10^{18} 次浮点操作的超级计算机, 其工作原理都是一脉相承的. 虽然上文列举的例子都经过高度简化, 但它们已经足够帮助你理解计算机系统中最本质的要素及其关联. 在后续学习的过程中, 我们将不断细化这些概念, 最终了解如何根据真实的ISA设计一颗真实的CPU芯片, 并在其上运行真实的程序.